

Guide to P?S/8 System Programs.

Copyright © 2015 »CLA Systems
CLASystems@gmail.com

Last edit: 30-Jul-2015 cj1

This document is a brief guide to the various P?S/8 system programs and keyboard monitor overlays as implemented in P?S/8 Version 8Z.

As an temporary accommodation, variations from this list associated with the 1976 release [Version 8P] will be noted.

For more information on any particular program [full lists of option switches, error messages, etc.] consult the various help files associated with the programs that have been released on the Internet at various PDP-8-oriented archive sites.

Note: This document revision only covers the TC01/TC08/TU55/TU56 release of P?S/8. Consult the various help files associated with additional programs that are specific to other implementations based on other hardware configurations. [Eventually, a long-form manual will be written to cover all programs in the basic system and the P?S/8 SHELL which is currently under development.]

Program Execution.

Note: For a more detailed discussion of command execution syntax, consult the **P?S/8 Keyboard Monitor/Editor Commands and Additional Information** document available separately. The notation used here is consistent with that document.

The basic P?S/8 system operates with a concise command mechanism that is the basis for various extensions used elsewhere. It is completely compatible with the command processor of COS-300/310 [since both P?S/8 and DIBOL-8/COS-300/310 are based on the R-L Monitor].

Execution of a system program is initiated in one of several ways:

`.R system-program:u [options]`

or

`.RU{n} system-program:u [options]`

or

`.system-program:u [options]`

The latter form can only be used if the program name is not in conflict with any of the two-letter commands of the keyboard monitor. As of the 1976 release of P?S/8, all standard system programs conform to this limitation.

`:u` refers to an optional logical device unit specification; system programs can be executed from any available logical device unit as long as it is a valid P?S/8 system at the same system release.

Depending on the system release, there may be certain system limits on the number of programs that can be used in the concise execution form. However, as a practical matter all relevant programs can be called up via the concise form. Any update to the system programs directory may exceed this limited capability; it is recommended that programs that do not interact with the operating system environment described by the keyboard monitor be placed in the rear of the directory to avoid any potential issues. [As of the 1989 release of P?S/8 Version 8Z, all system programs are contained within the first two pages of the directory; as such, no program is currently subject to this theoretical restriction.]

Optional command elements can consist of lists of input or output files, option switches and an `=xxxx` construct. [Note: Most programs demand the `=xxxx` parameter be expressed as an [up to] four-digit octal value in the range of 1-7776. Where noted, certain programs allow the value to be expressed in the current radix which could be either octal or decimal; in the latter case the value would be expressed as an [up to] four-digit value in the range of 1-4094]. [Note: For more information about command and file name considerations, logical unit considerations and additional information, consult the **P?S/8 Keyboard Monitor/Editor Commands and Additional Information** document available separately.]

Certain programs opt to self-parse any remaining characters on the line to support sub-commands specific to the program; these will be documented below as necessary. For these programs, all options as described here will precede special delimiters as required.

All execution commands must terminate with either `<CR>` or `<ESC>` [the latter will be indicated as `$`; the program determines if there is significance to the command delimiter; in most cases they are equivalent]. See Note 1.

System programs and specific program details.

Section 1: Overlays to the keyboard monitor/editor.

- 1) FIND [delimiter character] [search string] [optional delimiter character]

The stated string will be searched for in the file loaded into the edit buffer. The entire contents of the first line that matches the search string will be displayed on the system console. The keyboard monitor is completely reloaded to allow further editing commands.

Example:

```
.FIND "FOO"
```

```
1000 THIS LINE HAS FOO IN IT.
```

Options:

/A All matching lines will be displayed.

=xxxx Start the search at line xxxx instead of the lowest line in the file. [This line need not exist.]

Note: xxxx is stated in the current prevailing radix.

The use of a null string is allowed; as such, it is considered to be at the beginning of the line. [Using FIND with a null string and /A is equivalent to the LIST command.]

- 2) CHANGE [delimiter character] [search string] [optional delimiter character] [optional replacement string]

The stated string will be searched for in the file loaded into the edit buffer. The entire contents of the first line that matches the search string will be displayed on the system console with the matching string deleted and optionally replaced by the replacement string. The keyboard monitor is completely reloaded to allow further editing commands.

Note: If using a replacement string, the second delimiter character is mandatory.

Examples:

```
.CHANGE "FOO"BAR
```

```
1000 THIS LINE HAS BAR IN IT.
```

Section 1: Overlays to the keyboard monitor/editor [continued].

```
.CHANGE=1000 ""FOOBAR,  
1000 FOOBAR,THIS LINE HAS BAR IN IT.
```

Options:

/A All matching lines will be modified and displayed.

=xxxx Start the search at line xxxx instead of the lowest
 line in the file. [This line need not exist.]

Note: xxxx is stated in the current prevailing radix.]

If the search string is a null string, the replacement string
will be placed at the beginning of the line.

Note: Since null strings match all lines, it is strongly advised
that /A not be used and the =xxxx construct be used to specify
the [singular] desired line to be modified.

Note: The delimiter character is allowed within the text of the
replacement string.

Tip: All modifications are made to the contents of the edit
buffer. If the CHANGE command produces unwanted results, the
file can be reloaded to undo the unexpected changes.

3) ALLCAT

Normally, the P?S/8 keyboard monitor caches the entire Tiny File
System [TFS] directory for the current logical unit only. While
this is extremely useful on tape-oriented systems [such as
DECTape and LINCtape], commands within the keyboard monitor
involving multiple logical units still incur noticeable overhead
when attempting to perform file maintenance using common keyboard
monitor commands to load, save [with perhaps an alternate name]
or delete files from one logical unit to another.

All references to the TFS directories of every logical unit [0
through 7] are handled through a master caching routine located
in memory. [ALLCAT requires 8K memory]. Once any logical unit
is referenced, the appropriate TFS directory information will be
transferred through memory to allow all keyboard monitor commands
to access any file on any logical unit with dramatically
increased performance. If all appropriate file names are pre-
entered before attempting to copy the file contents[which is a
recommended technique], all tape motion will be limited to file
copy operations virtually eliminating any further access to the
TFS directories [with the attendant overhead] on both the source
and destination logical units.

Section 1: Overlays to the keyboard monitor/editor [continued].

The ALLCAT overlay inhibits all forms of additional program execution as a safety precaution. Complete exit of the keyboard monitor using the MO{nitor} command [or manual restart at 07600] will cancel ALLCAT operations; the restarted system will run normally.

Section 2: Text-related file utilities.

1) PRINT [*list of input files*] [options]

The contents of passed textual input files will be printed on the system line printer. Various options are available to format the files including the number of lines per page, various header options, accommodations to various forms of console devices [such as tear-offs for roll paper terminals such as the Teletype Model 33 ASR].

while the default device is the system line printer, if it is not available the output will be directed to the system console terminal. [Options exist to direct the output directly to the system console bypassing the system line printer.] See Note 2.

Section 3: Binary file and core-image related utilities.

Note: When considering core-image programs, all of field 0 is stored virtually in the storage space overlapping the positional files % and \$. While there are other uses for these files in other contexts, all such alternatives must be abandoned during any and all core-image related activities.

All extended memory operations are carried out directly.

Note: Other than Logical Console Overlay and/or extended device handler considerations, P?S/8 does not use the contents of extended memory; however, executing programs that access extended memory will tend to change extended memory contents. Should the contents of extended memory require preservation, programs that can change extended memory must not be executed or prevented from changing extended memory by use of the CORE command.

1) BIN [options]

The P?S/8 BIN program is actually a multiple function utility. It is primarily used to load the contents of binary files into memory. Many loading options are available including a choice of loader programs.

Note: with regard to binary file loading, a generic loader is always available; the user can opt to use a hardware-dependent alternate loader if implemented.

BIN is also used to read **BIN** [or **RIM**] paper-tapes for the purpose of creating P?S/8 binary files in the TFS directory. Both the low-speed and high-speed reader are supported.

BIN can also punch existing binary files to paper-tape in either **BIN** or [enhanced] **RIM** format. Both the low and high-speed punch are supported.

BIN can also be used to load a standalone version of the **BIN** and **RIM** loaders. A unique feature is the presence of a P?S/8 system bootstrap to be started at 07600 to allow P?S/8 to be reloaded after all paper-tape operations are over. [This assumes the P?S/8 bootstrap code is not destroyed by the consequences of running binary paper-tape programs.]

Chaining considerations.

BIN can be chained to from PAL for assemble-load-and-go operations. [The chain operation is inhibited by PAL if there are assembly errors.]

Section 3: Binary file and core-image related utilities [continued].

BIN can chain to MAP to display the load map of binary files created from binary paper-tapes. MAP can chain to BIN to load binary files passed to MAP for loading into memory [including a chain from PAL to create an assemble-map-load-and-go operation].

If initiated by PAL, the chain to MAP will be allowed; MAP will honor preventing the chain to BIN if there were assembly errors.

2) MAP [options]

The P?S/8 MAP program is used to create a load map of all binary input files passed to the program. The format is identical to the field dump within the TOPS10 PAL10 assembler and is a superset of the output of the OS/8 BITMAP program. Options are available to limit the mapping consideration to a selected subset of all possible fields.

Output of the MAP program is oriented to wide line printer output; options exist to limit the output on narrow terminals. The default output device is the system line printer. If the system line printer is not available, the output will be directed to the system console terminal. Options are available to direct the output to the console terminal even if the system line printer is available.

When MAP is chained from PAL, the output is cosmetically altered to resemble the previous listing pages from PAL; the printout is achieved in one continuous operation. If PAL options use the wide carriage output, MAP will automatically assume the comparable line width.

3) GET [*list of input files*] [loading options]

The GET command is used to setup the field 0 virtual area from the contents of the passed binary input files. The keyboard monitor is reloaded to allow further commands.

4) START [startup options]

The START command will run the latest core-image program stored in the field 0 virtual area. There are a few restrictions since the START command uses the /0-/7 option switches and the =xxxx command option to start the program at any specified location in 32K of memory; however, the system device can be write-protected beforehand as a safety precaution.

The R [or RU{n}] command used with either the % or \$ file name initiates temporary core-image execution. The starting address is 00200; all program options [other than program-dependent internal overlays] are supported. See Note 3.

Section 3: Binary file and core-image related utilities [continued].

5) ODT [options]

The P?S/8 implementation of ODT is a proper superset of all other ODT programs including the equivalent OS/8 command. It is completely "invisible" [as is defined in OS/8 terms, which implies small exceptions within the last page of any field where a breakpoint is defined, and the system kernel of 07600-07777 is not to be modified by any non-conforming program]. Commands exist to support multiple breakpoints and to display the contents of memory in a variety of octal and other formats.

Because multiple simultaneous breakpoints are supported, it is not possible nor necessary to "spoo" additional breakpoints. Most breakpoint mechanisms are implemented as JMP I 4; P?S/8 ODT uses JMS I 4 to determine the calling address of the latest sprung breakpoint.

Normally ODT does not require any command-line options. If no breakpoint operations have been performed, the command xxxxs [where xxxx is any octal value in the range 0000-7777] can be used to run the core-image program as a P?S/8 system program without restrictions. All options are passed as would apply to a system program without exception. [This is the preferred method for using a core-image as a system program if the restrictions caused by the START or R or RU{n} % or \$ form are unacceptable. However, once a breakpoint has been setup, ODT modifies most of the passed parameters.

6) BSAVE [*list of output files*] [save options].

The BSAVE utility saves specified areas of memory into binary output files. Any form of memory range can be specified from a single location to an entire memory field.

Example:

```
.BSAVE FOO,BAR,FOOBAR < 0-7577,10001,20200-20377
```

P?S/8 binary format approaches 6/7 the efficiency of core-image without imposing block loading. Each file is 2048 12-bit words including a descriptor word for each six loading words. Up to 17 output files can be created in any one command; this corresponds to the maximum number of input files that can be loaded by BIN for any one binary file loading operation. [About 28K is allowed per command; if necessary, BSAVE and BIN operations can be performed in two memory sections if 32K must be loaded.]

Section 4: Programming utilities.

1) PAL [options]

The P?S/8 implementation of the PDP-8 Assembly Language [PAL] is a superset of all other known fully-compatible PDP-8 assemblers [to within nitpicks]. It is highly modular in implementation; certain options can be deleted to allow additional symbol table space or improve performance through the use of larger input/output buffers.

Note: This is especially crucial on small systems where less than 7K memory is available. PDP-8 assembly programs vary greatly in symbolic content. The largest symbol table possible is 4096 symbols including permanent symbols and internal directives [pseudo-operations].

Notable features include the optional support of current page, page zero and conditional literals [the latter a unique feature of P?S/8 PAL]. When enabled, P?S/8 PAL is a superset of the dual-mode LAP6-DIAL assembler for PDP-8 and LINC mode assembly. [The DIAL assembler does not support literals.]

P?S/8 PAL can take full advantage of wide-carriage printers [using available option switches]. The default output device is the system line printer. If the system line printer is not available, listing output defaults to the system console terminal. [Options exist to direct the output to the system console bypassing tests for the system line printer.]

An extremely useful feature of P?S/8 PAL [also supported by the PAL III paper-tape system assembler] is the ability to obtain a single-pass assembly. At the end of pass one, all symbols still undefined are printed on the system console referencing the first location within the assembly where the undefined symbol was referenced. When compared to other assemblers [such as PAL10 and PAL8 which do not implement this], this feature can be virtually indispensable in the early stages of program development. [Other systems offer no productive method of remedying this predictable program development situation other than wastefully producing an entire listing including all references to each and every undefined symbol; this could amount to the tedious process of manually looking through thousands of lines referencing all of the undefined symbols instead of merely looking at a compact list of the undefined symbols by name in a well-ordered listing.]

Unlike many other PDP-8 assemblers, P?S/8 PAL performs either a one pass or a two pass assembly. As required, binary output and listing output are created simultaneously.

Section 4: Programming utilities [continued].

Note: There are anomalies of PDP-8 assembly when certain features are used improperly that create output that changes for each assembly pass. As such, three pass assemblers such as PAL10 and PAL8 can create a subtle anomaly condition where the binary output does not match the assembly listing. By using a two-pass approach, this is impossible; the debugging process to locate the incorrect usage will be greatly improved.]

P?S/8 PAL can chain to MAP and/or BIN to create automatic assemble, [map,] load and go operations.

Note: Extremely long source files can be created through the use of extended-length files [such as those created by the OS8CON program]. See Note 4.

2) FOCAL [options]

P?S/8 FOCAL is a binary-compatible superset of FOCAL, 1969. Virtually all known user-created patches to FOCAL, 1969 can be used with P?S/8 FOCAL [and a few have been added internally to be enabled/disabled by command-line option switches].

The functionality of FOCAL is vastly improved as custom binary files can be passed to create extensions to the language. [The process of customization is described in large part in FOCAL-17, which is a manual on this subject distributed by DECUS; known copies exist on various PDP-8 archive websites.].

A total of 17 input files can be passed to FOCAL [all binary patch files must precede all text files]. As FOCAL processes input, the contents of all text files replaces the normal console keyboard device; once the end of the last file is reached, input control returns to the console using a new fully-buffered console routine.

Note: The original FOCAL, 1969 only supports fully-buffered console output. The new routines largely avoid the problem of input overflow on the low-speed reader if ASCII paper-tapes are read into FOCAL directly; however, it is always recommended the paper-tapes be converted to P?S/8 text files. See Note 5.

Because of the ability to allow FOCAL to interact in various ways with P?S/8, it is possible to define complex projects that would never be thought of as viable in the paper-tape environment of FOCAL, 1969.

Section 4: Programming utilities [continued].

Many project examples are available that use FOCAL in various ways [such as allowing FOCAL programmers to control assembly language projects as a "front end" to tweak control parameters] or outputting display graphics to a variety of devices or to process indefinite lists of data for statistical purposes, etc. Numerous biomedical programs have been written using P?S/8 FOCAL at the heart of the overall application.

There is a command-line option to suppress console output during the time FOCAL input is obtained from P?S/8 text files [which is recommended during very long number-crunching sessions using data obtained from a long list of input data contained in P?S/8 text files].

P?S/8 FOCAL supports the 4-word floating point modification and the 8K patch to place all FOCAL text storage into field 1. [Custom versions of these files are available as P?S/8 binary files; the original versions cause certain cosmetic problems solved by using the P?S/8 variants in a proscribed order.]

While P?S/8 FOCAL does not directly support the original FOCAL initial dialogue [INIT], the configuration questions asked have been implemented as command-line option switches.

Note: The restriction regarding the requirement that one of the trigonometric packages can only be retained if the other package is also retained has been removed by performing dynamic relocation; this was never offered in the original FOCAL, 1969.

A FOCAL, 1969 bug [fixed in FOCAL-12 and FOCAL-8, neither of which are binary-compatible with FOCAL, 1969] has been fixed in P?S/8 FOCAL without losing overall binary compatibility.

Note: Unlike FOCAL-8, P?S/8 FOCAL does not attempt to internally self-sabotage if running on a DCC-112 or DCC-112H computer.

A report-only version of the initial dialogue may be developed as a binary patch file to P?S/8 FOCAL. [No interactive questions can be asked as all relevant decisions are made by command-line options; however, a user application written in FOCAL might want to start with a configuration report including the status of the 4-word and 8K options and the trigonometric packages, to be used as a documentation aid. A complete command-line can be reconstructed including the names and types of passed files.]

While small changes can be made to the FOCAL program code from within FOCAL, it is strongly recommended the file be maintained within P?S/8 text files. The keyboard monitor ED{it} command is patterned after the FOCAL MO{dify} command and is an analogous superset of the same functionality.

Section 5: Additional file maintenance utilities.

Note: P?S/8 files can be maintained on any P?S/8 system on an individual basis using the keyboard monitor, perhaps enhanced by the ALLCAT utility.

1) FILMAN [options]

The P?S/8 FILMAN [file manager] utility allows for patterned copy or deletion of groups of TFS files with a single command. Various wild-card options are supported to allow the creation of powerful operations.

FILMAN operates in both a command-line mode and an interactive mode [as determined by a single command-line option switch which bypasses the general command-line mode].

Command-line options allow the optional filtering of file types if desired; the default is all files will be considered. All files can be selected automatically or be subject to approval by manual input.

Future releases of FILMAN may include various additional options such as a MOVE, RENAME or UNDELETE facility. See Note 6.

A future direction of development of the FILMAN utility may include aliased program named. This may be a challenge since certain proposed options are in conflict with present keyboard monitor commands such as MO{ve} and RE{sequence}. [UNDELE{te} and COPY do not have this potential; however, a complete set of alias entries would be helpful. [One way to get around this is to use alternates such as MV and RN such as are used in unix systems.] Due to interaction with these commands and other issues, this entire issue may be tabled pending potential changes in the relevant keyboard monitor commands.

Note: There is already an open issue regarding the MO{nitor} command suggesting changing the command name to SY[stem} or BY{e} to encourage intuitive acceptance from those familiar with implementations of BASIC where these are the prevailing command names. MO{nitor} is the current command name and is compatible with the R-L Monitor System command for the same function.

Section 6: User-settable configuration programs.

1) SET [options]

The SET utility is used to configure a variety of custom keyboard monitor and other functions that affect various aspects of using P?S/8. The /A option can be used to view all available options. In certain cases the default values for the various settings will be displayed.

Some of the settings determine how the keyboard monitor interacts with various supported console devices such as the VT05 and VT52 display terminals, as well as hard-copy devices such as the Teletype Model 33 and Model 35.

Note: A complete list of all options is available in the help file for the SET program that is distributed on various PDP-8-oriented archive websites. See Note 7.

2) CORE [options]

The CORE command with options given displays the current memory configuration of P?S/8.

The CORE ALL command will maximize all parameters [if possible]. Assuming additional memory is available, it is possible to limit access to additional memory by setting an explicit upper limit using CORE 0 through CORE 7 as required. An error message will be issued if an attempt is made to enable non-existent memory.

Regardless of the setting of the CORE command, if the Logical Console Overlay is enabled, it will be loaded into [at least] the highest available memory field.

Note: To date, no configuration of the Logical Console Overlay actually uses more than one memory field; this remains a largely theoretic consideration.

For specific configurations of P?S/8, the system device handler requires the use of an extended memory section [loaded during the bootup process]. This will always be located in locations 6000-7777 of the highest memory field available. When this is in effect, it is not possible to obtain access to the highest memory field. However, if the Logical Console Overlay is not enabled, locations 0000-5777 may be available to certain programs such as PAL, which will use available memory space to support more symbols during the assembly process. As such, certain 8K systems can support a 7K assembly with significantly more symbols than can be processed in 4K of memory. [PAL will always use completely available memory fields to allow assembly of programs with up to 4096 symbols.]

Section 6: User-settable configuration programs [continued].

[Depending on release specifics, several other system programs can take advantage of additional memory either as a partial or full memory field.]

3) DATE [options]

The P?S/8 DATE command can be used to set the system date. Unlike OS/8, the date setting is permanent until changed again. When the DATE command is used without any options, the current system date will be displayed.

Certain specific dates have associated *message-of-the-day* information. A limited ability to add user-preference additional messages is available [which cannot override the permanent messages].

Certain system programs support current date information in the associated output. See Note 8.

The implementation of the DATE command is currently under development. See Note 8.

4) CONSOL{e} [options]

The Logical Console Overlay can be enabled or disabled by the following commands:

CONSOL{E} /E This enables the Logical Console Overlay.

CONSOL{E} /D This disables the Logical Console Overlay.

See Note 9.

The configuration of the Overlay can be updated by passing binary files that are specially written for the purpose. While these files are ordinary TFS binary files, they are heavily checked for compatibility with the Overlay mechanism to prevent internal corruption.

Configurations are available for a variety of alternate console devices such as console terminals with device codes other than the standard 03/04 [which can allow a remote console terminal]. The VT8E is a featured device and includes special display emulation designed for use with the keyboard monitor.

Additional devices can be supported such as the VR12/VR14 on the PDP-12.

Section 6: User-settable configuration programs [continued].

A unique feature of the Logical Console Overlay is interaction with the system device handler on a customized basis. Each system device handler provides error and/or performance intercept routines to allow the Overlay to improve performance in specific ways [such as implementing dynamic initial search direction support on PDP-12 LINCtape systems] and in all cases the Overlay interacts with detected device errors. This allows the user to make the required decisions with regard to the disposition of device errors.

Note: Users of MS-DOS may recognize this as the precursor to the familiar *Abort, Retry, Fail* dialogue, but actually it has its roots in the ADSS/9 system IOPS4 support.

On the LINC-8 with 8K, the Overlay can use the small oscilloscope display for handling device errors; it is somewhat impractical for a complete console display. The PDP-12 supports half-size characters which provides a more resolute display. On devices that use terminal emulation such as the VT8E, the screen contents will be changed to the device error handler; when the error is dismissed, the normal terminal screen returns.

The Logical Console Overlay also configures the system line printer for all Overlay sessions. A variety of devices can be supported including serial printers that require reverse channel Control-S/Control-Q support and/or character buffering.

Note: OS/8 cannot drive a parallel line printer to faster than about 300 LPM regardless of printer capability. The P?S/8 Logical Console Overlay provides buffering that allows at least 1200 LPM on printers capable of high-speed output when properly buffered. Most programs lack the sophistication to drive a line printer beyond that obtained by simplistic output routines.

A logical interrupt handler is available to interface the Logical Console Overlay with interrupt-driven programs such as P?S/8 FOCAL. By calling the logical interrupt handler first, logical devices can be handled on an interrupt basis correctly. All input and output to the console device and the line printer [and reverse channel support] are handled.

The default configuration for the Logical Console Overlay uses the device 03/04 standard console device and the device 66 standard line printer without reverse channel support [with internal line printer buffering]. This will function on all system configurations and will enable the interaction with the system device handler.

Note: Since configuration updates are permanent, binary files are available to revert the configuration to the standard release.

Section 7: Batch and advanced system utilities.

1) BATCH [options]

P?S/8 BATCH runs on all P?S/8 configurations as an overlay to the keyboard monitor. Command input is taken from the contents of passed input files. A command-line option enables checkpointing of BATCH operations preserving internal status across system restarts; this allows fully automated system operations. [It is possible for BATCH to initiate another BATCH operation, in which case the system can be put into a deliberate perpetual loop. This can be useful for certain purposes such as running a group of diagnostics merely by initiating the overall process.]

Nearly all system programs can run under BATCH in an identical manner to the way they are initiated manually. For the few instances where there are issues [which invariably are overlay commands to the keyboard monitor itself], BATCH must be initiated with the checkpointing disabled. [Under certain circumstances, the checkpointing-disabled mode of BATCH can itself be run under the checkpointing variant of BATCH. Thus, even these few problematic programs can be run under BATCH as long as all considerations are met.]

The only hard restrictions under BATCH operations are certain keyboard monitor commands and operations requiring control characters [such as the ability to rewind and unload the system device and the ED{it} command which is controlled by additional control characters specific to certain ED{it} operations].

It is possible to perform file edits from BATCH. A file can have lines updated and saved [perhaps as a temporary file]. Conditional assembly can then be performed that depends on small edits to control files changing the outcome or scope of the assembly process.

Several keyboard monitor commands have been added which are BATCH-oriented such as the PL{ease} command and the ^^ command. By carefully controlling the state of ECHO [using the EC{ho} command], the output produced during a BATCH session can be carefully controlled [and cosmetically enhanced].

The most conventional usage of P?S/8 BATCH is to create a file containing any mix of keyboard monitor commands and program execution. This file can be passed to BATCH as an input file. All of the commands will be carried out in sequence. [/E is required if program execution is used with additional command lines contained in the BATCH control file.]

Section 7: Batch and advanced system utilities [continued].

2) DUMP [options]

P?S/8 DUMP is a small versatile program to display the contents of any block on any logical unit of the system device. It is also possible to transfer a group of blocks from any block on any logical unit to other blocks on any logical unit. The latter operation must be done cautiously as serious damage can occur to the system if the wrong blocks are accessed!

DUMP can also modify the contents of any block. Caution must be observed as serious damage can occur to the system if incorrect modifications are made.

DUMP can also take control files to automate DUMP operations. See Note 10.

3) BLKODT [options]

P?S/8 BLKODT allows access [and optional update] to any logical block on any logical unit using the command environment of ODT [where applicable; there are no execution or breakpoint-oriented commands]. As in the case of P?S/8 DUMP, BLKODT should be used carefully as making incorrect modifications can cause damage to the system.

Unlike DUMP, BLKODT can display the contents of blocks in a variety of formats. Output format control is identical to the P?S/8 ODT implementation. See Notes 11 and 12.

4) BLKCPY [options]

P?S/8 BLKCPY is a universal block-oriented copy/compare program. All blocks are addressed by using P?S/8 non-system device handlers.

Note: Currently, P?S/8 BLKCPY is the only system program that makes use of non-system device handlers.

Any block on any supported device can be copied [or compared] to any block on any other supported device. The syntax is similar to command-decode lines as used in OS/8 except all references are to absolute blocks instead of files. Blocks can be selected either by specifying a range of blocks [starting through end block] or by a starting block and an =xxxx expression as the count of blocks to copy.

Section 7: Batch and advanced system utilities [continued].

Command syntax uses either `<` or `>` to indicate the direction of the transfer. The user can choose which ever "direction" of the copy operation is more meaningful. Unless a command-line option is given to suppress comparison, all blocks copied are read-verified at the individual word level.

BLKCPY can also perform an absolute compare on groups of blocks by using `!` as the separator [in which case no blocks are written]. See Note 13.

BLKCPY always attempts to allocate maximum available memory to improve throughput. If 12K of memory is available, BLKCPY can copy one DECTape to another faster than by any other method including the standard DTCOPY program [which cannot verify the copy operation].

BLKCPY can be automated by passing control files containing BLKCPY commands.

Future implementations of BLKCPY will work with the new double-precision non-system device handlers planned for use with the P?S/8 SHELL to allow humongous device support.

Section 8: System utilities that create reports.

1) DIRECT [options]

P?S/8 DIRECT is a utility program to report the contents of TFS directories on specified logical device units. Various file filters are available to tailor the output if desired. Optional enhancements can show specific file type if desired independent of optional file filters chosen]. See Note 14.

DIRECT can also output complete information about all current system programs including specific internal information [some of which will be dropped from support in future releases due to the development of the P?S/8 SHELL]. The output can be formatted in a variety of ways and can be directed to the system line printer [if available] or the console terminal. See Note 15.

2) SYSTAT [options]

P?S/8 SYSTAT outputs a variety of system configuration parameters including the availability of EAE and the CPU type. The status of the Logical Console Overlay is reported both with regard to its internal configuration as well as whether it is enabled. See Note 16.

Note: Certain items reported in SYSTAT are being phased out due to the development of the P?S/8 SHELL.

Section 9: Conversion utilities.

1) OS8CON [options]

P?S/8 OS8CON is a bi-directional conversion program to either read or write OS/8 file data on media that can be read by the system device handler [such as DTA1: which is logical unit 1 of the P?S/8 system device handler for DEctape]. See Note 17.

P?S/8 input files can be any mix of TFS text files and extended-length text files as necessary. The OS/8 target can be specified in terms of command-line options to select the logical unit and the =xxxx construct which specifies the starting OS/8 record [NOT the P?S/8 logical block].

Converting to P?S/8 output files is selected by specifying output files instead of input files. A list of TFS output files can be specified. By the use of a specific command-line option, a list of extended-length files can be used for extremely large files.

After all conversions, the count of output files created is reported. [In the case of converting to OS/8 format, the count is always one.]

No OS/8 directory operations are performed. Assuming the file is written to an otherwise empty OS/8 device [such as DTA1:] the following OS/8 PIP commands can produce a proper directory entry:

```
.R PIP
*DTA1:</Z
*DTA1:FOOBAR.PA</I=xxxx  xxxx is any large record count [1000]
*DTA1:FOOBAR.PA<DTA1:FOOBAR.PA/D/A
```

2) L6DCON [options]

L6DCON is a bi-directional conversion program to either read or write LAP6-DIAL/DIAL-MS text files on 256 words/block LINCTapes. All program usage is modeled on the OS8CON program.

The default DIAL parameters are logical unit 1 and the default block where the DIAL edit buffer is located. It is recommended to use absolute block 0000 on an empty DIAL tape in case the file exceeds DIAL's maximum editable file size.

Note: If the file will be edited in the DIAL environment, the file must be broken down into small sections, and the DIAL assembler CHAIN option will be required to actually assemble the composite text of the original source code.

Section 9: Conversion utilities [continued].

No DIAL directory operations are performed; however, DIAL includes commands to directly access programs by absolute starting block.

Note: The development of DIAL-MS was largely performed using DIAL10, a variant of PAL10 that only runs on the PDP-10 TOPS-10 operating system. Like the DIAL assembler itself [as well as the P?S/8 PAL assembler], DIAL10 supports dual mode PDP-8 and LINC mode assembly and is 100% compatible with DIAL source code files. Particularly long files were edited only on TOPS-10 then brought to the DIAL environment for the purpose of producing binary output for testing. The obscure feature of direct access by starting block number was retained in released versions of LAP6-DIAL and DIAL-MS.

L6DCON is only available on PDP-12 and LINC-8 systems; internal device handlers are used as required.

Section 10: Miscellaneous utilities.

Note: Depending on the system device and/or intended potential system target, additional hardware-dependent programs may be supplied. The programs listed here are generally supplied on TC01/TC08/TU55/TU56-based systems only.

1) DT4MAT [options]

DT4MAT [also known as DTFRMT] is an adaptation of the TOG-8 DECTape formatting program from DEC. It is slightly enhanced over the standard DEC version to allow the creation of custom-length DECTapes; the accepted alternate standard length is 3300 blocks which compares well with the standard length of PDP-8 LINCTapes as used on the PDP-12.

Note: Only premium DECTapes can achieve this extended length; at least 40% of all DECTapes will not be able to be formatted to this length, but many tapes in good condition will pass certification.

Non-standard lengths are achieved using a command-line option. See Note 18.

2) DTCOPY

DTCOPY is the standard DECTape copying program as released by DEC. It can copy a complete PDP-8 or PDP-10 [and all other formats as used on other machines that are compatible with this tape format] in 4K of memory using a three-pass scheme where the tapes are read and written on all six phases of tape motion. [Certain blocks are read and written backwards.]

3) RK4MAT

RK4MAT [also known as RKFRMT] is the standard DEC formatting program for the RK8E/RK8F/RKS8E controller and RK05/RK05F drives; certain patches have been made to allow operation on pre-Omnibus machines with the RK8F/DW8E. It is provided as an accommodation to systems with only a single RK05 drive as well as DECTape to allow the RK05 drive to more easily be unloaded and then mounted with new media that requires reformatting. [P?S/8 RK-based systems always include this utility as a standard complement of that configuration.] See Note 19.

4) Additional utilities are provided on various P?S/8 configurations generally to support hardware-dependent utilities such as disk formatting utilities. See Note 20.

Footnotes.

- 1) The 1976 release of P?S/8 [Version 8P] does not support the <ESC> character as a command delimiter.
- 2) The 1976 release of P?S/8 [Version 8P] does not completely support all options of the PRINT command.
- 3) The R or RU{n} commands when used with % or \$ [to execute the field 0 virtual image] are not available in the 1976 release of P?S/8 [Version 8P].
- 4) Extended-length files can be created by the OS8CON program from OS/8 format text input. If necessary, the OS/8 format can be used as an intermediary format for creating an extended-length file from a stream of TFS text input files as a first step.

In the case of the 1976 release of P?S/8 [Version 8P], the OS/8 format text data can be created by either the CONVRT or BIGCON programs.

However, the creation of extended-length files in the 1976 release of P?S/8 [Version 8P] can only be accomplished by a crude method using BIGCON. This involves assigning a specific TFS file output file to be considered the designated file name of the extended-length file. The rest of the storage on the designated device is used to store the extended-length file.

This is a hazardous operation and requires the front panel switch register set to the limiting value for extended-length file creation. For example, it would be recommended to use a logical unit where the first [and only] file name in the directory will be used to create and access the extended-length file.

For a standard DEctape-based system, the switch register should be set to 2700. Assuming the only entry in the TFS directory is named EXTFIL and the first available block for the TFS directory on that device is 0500, this would mean the extended-length file for that logical unit could be as large as 0500-2677; maximum lengths must be set to whole multiples of 10 octal blocks.

For truly humongous OS/8 source file conversion, multiple extended-length files may be created on multiple logical units. Regardless of system release or access to the extended-length files, up to eight extended-length files can be passed to P?S/8 system programs [presumably P?S/8 PAL for the purpose of an extremely long assembly].

Footnotes [continued].

All known PDP-8 programming projects require less storage than this provides; the longest program in the experience of the author of this document [the principal developer of P?S/8] would require three such extended-length files. This would be roughly the equivalent of 48 TFS text files for a single assembly.

The source files of P?S/8 were developed using PAL8 during a significant portion of its overall development timeframe which spans several decades; early development made use of PAL10 on a PDP-10 TOPS-10 time-sharing system.

At no time after an early release of P?S/8 PAL [prior to the 1976 release of P?S/8] became available as the then-current release of P?S/8 PAL, was P?S/8 PAL not up to the task of assembling all portions of the then-current P?S/8 source code; however, various improvements have been made over the years that affected the assembly of even larger projects.

In one notable case, the source code had to be radically modified to allow OS/8 PAL8 to create slightly incompatible output due to lack of support for conditional literals; infrequently, manual effort was expended to synchronize the source code by observing the output generated by P?S/8 PAL. Conditional assembly methods were used to force the output from PAL8 to be binary-consistent with the P?S/8 PAL output by the use of statically-determined page zero literals instead of conditional literals.

Since PAL8 can only work with static definitions to create page zero literals, the source code [when assembled by PAL8] is always in danger of being inconsistent with the programmers' intentions; this project could not have been successfully developed without P?S/8 PAL.

Note: Conditional assembly can be written to allow program sections to only assemble on P?S/8 PAL; as such, certain sections of this program were written for separate PAL8 and P?S/8 assembly as required. Only one master source file is needed when this technique is used.

Still newer releases of P?S/8 [projected to be first released as part of P?S/8 Version 9A] will change the internal directory structure to streamline the support for extended-length files as described here. A vestigial effort was once attempted to allow multiple extended-length files, but this was never completed.

Since the P?S/8 SHELL is now contemplated to perform this function and much more [and will have its own unified directory structure], the remnants of the vestigial support will be removed from the basic P?S/8 system.

Footnotes [continued].

Future releases will support one extended-length file per logical unit. Extended-length file name maintenance will be added to the SET utility program.

Note: Presently extended-length file names must be created manually using P?S/8 DUMP or BLKODT. To prevent file name anomaly, the extended-length file should be set to a unique name, as any TFS file name on the same logical unit will always have preference. While extended-length file renaming is seldom required, the proposed method will allow any level of P?S/8 user the ability to perform this minimal function. Once named, extended-length files are passed to system programs in an identical manner as TFS files.]

- 5) The 1976 release of P?S/8 [Version 8P] has limited support for ASCII text paper-tapes by use of the HR{reader} command. This facility was removed in later releases.

Since paper-tape has had a resurgence of popularity [often because this is the only way to pass text files to P?S/8 when running the SIMH simulator and wishing to avoid using OS/8's paper-tape device handler as part of an intermediary solution], a simple utility [compatible with all releases of P?S/8] is contemplated to more effectively solve this problem, which is tentatively known as PTUTIL. For more information on PTUTIL, consult Appendix D of the **P?S/8 Keyboard Monitor/Editor Commands and Additional Information** document available separately.

- 6) The 1976 release of P?S/8 [Version 8P] does not support the FILMAN utility; certain necessary internal features of the keyboard monitor were not available until developed years later.
- 7) The 1976 release of P?S/8 [Version 8P] uses the VARY program to perform most of the functions of the SET utility; the command syntax is generally similar [if applicable]. VARY /A can be used to show all options implemented.
- 8) Both the 1976 [Version 8P] and 1989 [Version 8Z] releases of P?S/8 date support an algorithm that provides for an 11 year range of dates passed in memory as an internal parameter of the system kernel. System dates are ordinarily unique to within any date within an eight-year period; by providing the anomaly of this method, adjustment of the base year [originally set to 1960] could be deferred by nearly four years after the system changes could first be made.

Footnotes [continued].

The implementation of this method was at an early stage of development when the 1976 version of P?S/8 [Version 8P] was created. As such, the DATE command prevents setting any date outside of the range of 1976 through 1986.

By the release of P?S/8 Version 8Y, all system programs that conform to the date mechanism define internal storage locations allowing adjustment of the base date; this allows manual patching of any date range from 1960 through 2087.

Future releases of P?S/8 will abandon this method entirely. The new base date is 01-Jan-1900 and the entire date is available in the system kernel in an 18-bit format. As such, all programs will be modified to obtain the date directly without restriction.

By enlarging the supported date range to 1900 through 2411, all relevant file dates from the twentieth century well into the future [as of this writing] are supported.

While basic P?S/8 TFS files do not support file dates, the P?S/8 SHELL directory will fully support the expanded date support. As such, P?S/8 date support will be more useful than OS/8, unix and other systems, especially for existing computer files from the 1960s and the 1970s [which cannot be supported by most of the "modern" operating systems in use today].

- 9) The 1976 release of P?S/8 [Version 8P] only supports the VT8E and cannot be configured. The program is managed by use of the VT [ON] or VT [OFF] commands as required.
- 10) The 1976 release of P?S/8 DUMP does not support control files.
- 11) The BLKODT user interface is available for 256 words/block LINCtapes using a derivative program called LTBODT. This program can only run on the PDP-12 or LINC-8 and uses internal handlers to determine the appropriate access to 256 words/block LINCtapes.
- 12) BLKODT is not available on the 1976 release of P?S/8 [Version 8P].
- 13) BLKCPY is based on SYSCPYP which can only support the system device and all logical units. For backward compatibility, BLKCPY has a pseudo-device called SYSx: [where x is the desired logical unit in the range of 0 through 7] to allow the former operations [by indirectly calling the system device handler].

BLKCPY is not available on the 1976 release of P?S/8 [Version 8P]; however, a very early [and incompatible] release of SYSCPYP is supported.

Footnotes [continued].

- 14) The DIRECT utility is not available on the 1976 release of P?S/8 [Version 8P].
- 15) In the 1976 release of P?S/8 [Version 8P], certain information about available system programs is obtained using the SYSTYP program which was dropped in future releases in favor of the DIRECT program used with the /S option].
- 16) The SYSTAT utility is not available in the 1976 release of P?S/8 [Version 8P].
- 17) OS8CON is not available in the 1976 release of P?S/8 [Version 8P]. CONVRT and BIGCON are used as necessary to accomplish the same basic functionality.

OS8CON is based on the design of both of these programs with newer support for extended-length files not available in the 1976 release. For more information, see Note 4 above.

- 18) The 1976 release of P?S/8 [Version 8P] only supports the standard DECTape formatting program.
- 19) The 1976 release of P?S/8 [Version 8P] does not including the RK4MAT program.
- 20) The surviving 1976 DECTape release of P?S/8 includes several programs that are not part of the original release.

UCNVR8 is a program written by Mark J. Hyde of Syracuse University to convert files on DECTapes between various systems including P?S/8, OS/8, ADSS-9/15 and possibly the Disk Monitor System on DECTape. It runs in 4K memory as a stand-alone utility and only supports the TC01/TC08/TU55/TU56 DECTape. [If run on the PDP-12, the TC12F option is used to access DECTapes; LAP6-DIAL LINCTapes are also supported using the DIAL default edit buffer only.]

UCNVR8 uses a unique form of DECTape support where the motion bit can be maintained; this is useful when accessing PDP-9/15 DECTapes where the next block to be read can be accessed without tape turnaround.

UCNVR8 is a stand-alone utility and need not be a P?S/8 system program. It can be made more generably available for use on newer releases of P?S/8 or OS/8 [or LAP6-DIAL/DIAL-MS].

P?S/8 TFS text files created by UCNVR8 are deficient in that the internal line numbers and text pointers are not created; however, they are properly accounted for thus each file is properly allocated without overload.

Footnotes [continued].

The NUM program was also written by Mark Hyde as a partial remedy to this problem. Files are passed to NUM to be read in and corrected to proper TFS standards then written back to the same storage blocks.

Note: The NUM program is ignorant of P?S/8 output file conventions; the first file in the passed parameter list is used without regard for input or output status. To within this proviso, NUM can be run on any P?S/8 release.

Charles Lasner pointed out the NUM program is superfluous as the same results can be achieved through the use of a short BATCH program. Assume the file FOOBAR must have line number entries created. Create a control file FOO with the single line:

```
100 SEQUENCE
```

Note: This assumes the 1976 release of P?S/8 [Version 8P]; for all newer releases use LNUMBER]

```
.BATCH FOO FOOBAR
```

This will put the keyboard monitor into automatic line numbering mode; the text contents of the file FOOBAR will be used to create a complete file in the edit buffer.

When the BATCH file finishes, type control-D for the 1976 release [or control-Z for newer releases] to end the automatic line-numbering mode. If successful, the edit buffer can be saved to any desired file name on any logical unit.

The TRAN program was used to support the first application of the InterProcessor Buffer [IPB] dual-bus asynchronous transfer device. A PDP-8 with a high-speed reader provided BIN loader support to another PDP-8 with only a Teletype Model 33 [and no storage device]. [The Syracuse University PDP-8 computer lab had an odd collection of machines including one with neither a paper-tape reader nor a DECTape storage device. By loading into the receiving computer a BIN loader program [with modified instructions to match the device code of the IPB, this machine was more useful in the shared environment.

TRAN is a stand-alone program that can be loaded by any operating system. [TRAN was run from P?S/8 on a 4K PDP-8 with TC01/TU55 DECTape and the high-speed reader/punch and the server side of the IPB.]

Footnotes [continued].

Charles Lasner used the IPB to allow a PDP-8/E equipped with a VT8E to control a LINC-8 by modifying PROGOFOP to support the IPB instead of the device 03 keyboard of a Teletype Model 33. [On the LINC-8, pure LINC mode programs do not support the teleprinter.]

Eventually, the IPB was used to provide device support between two machines that have different storage devices; this allowed each to have access to the storage device of the other. A common server program was written to be used on the storage-providing [server] side of the dual interface while device handlers were used on the other side.

Note: There are bootable IBP-based systems for P/S/8 [and OS/8 can also be supported on 8K or larger machines]. Had this system been available at the time the TRAN program was in use, TRAN would never have been required.] [Non-system handlers also exist for OS/8 IPB access.]

It is hoped the IPB becomes the inspiration for a modern multi-purpose device for the PDP-8 to provide mass storage as well as potential additional devices using a support computer or "smart" microprocessor.

[End-of-file]